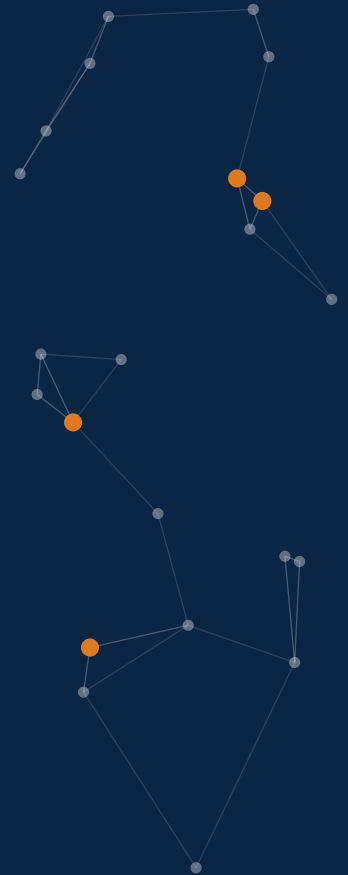


Real-Time Identification of Fraud-Linked Cryptocurrency Exchanges

Implementation plan — backend and frontend, step by step, built entirely on free APIs and public datasets



TRON-first

Multi-hop tracing

VASP attribution

Explainable risk

Live alerts

Freeze notices

PREPARED FOR

Gauri Jain and team (6 members)

PROBLEM OWNER

Ministry of Home Affairs — Indian Cyber Crime Coordination Centre (I4C)

TEAM STACK

React + Tailwind · Node.js/Express + Prisma · PostgreSQL · Neo4j · Redis · Python (ML) · Docker

VERSION

1.0 — 20 September 2026

CONTENTS

What's in this plan

1 Executive summary 3

2 System architecture 4

3 Free APIs and providers 5

4 Free and public datasets 6

5 Backend implementation (B0-B11) 7

6 Frontend implementation (F0-F9) 13

7 API contract between frontend and backend 15

8 Team split and six-week timeline 16

9 Demo script and metrics 17

10 Risks and mitigations 17

A Address and hash validation rules 18

B Features for the risk model and their reason text 18

C Repository layout and environment 19

D Source links 19

How to use this document

Sections 1–4 are the shared foundation: what you are building, the architecture, and every free API and public dataset with its current free-tier catch. Section 5 is the backend build in 12 ordered steps (B0–B11); Section 6 is the frontend build in 10 steps (F0–F9). Every step lists its goal, concrete tasks, the free resources it uses, and a testable “done when” check, so each teammate can pick up a step and know when it is finished.

Sections 7–10 cover the API contract between the two halves, how to split the work across six people over six weeks, the demo script and metrics judges will ask about, and the risks most likely to bite you.

SECTION 1

1 • Executive summary

The platform turns a victim-reported wallet address into an actionable lead for the investigating officer. It ingests complaints and auto-detects the chain, giving TRON/USDT-TRC20 a dedicated fast path; walks the money outward hop by hop across TRON, EVM chains and Bitcoin, storing every hop in a graph database; names the nearest exchange (VASP) using open label sets and clustering heuristics while flagging mule and layering wallets; scores every wallet with a hybrid rules-plus-ML model and explains that score in plain language; watches suspect wallets live and raises an alert the moment funds reach an exchange (the freeze window); and produces a freeze notice and an evidence report as PDF and JSON, stamped with an integrity hash.

Everything runs on free tiers and public data. Two inputs are not publicly available — the NCRP/CFCFRMS complaint feed and the SAHYOG portal — so the plan builds them as swappable adapters backed by mock servers with a written contract. When I4C grants access, integration becomes a configuration change rather than a rewrite. That is the honest way to pitch it: **integration-ready**, not integrated.

Key design decisions

Decision	Choice	Why
Core API	Node.js + Express + Prisma + PostgreSQL	Your team already works in this stack, so week one goes into features, not learning.
Chain vs ML split	Node for all chain I/O; Python (FastAPI) only for ML and XAI	Chain SDKs (TronWeb, ethers, bitcoinjs) are JavaScript; XGBoost + SHAP tooling is Python. Keep Python small and stateless.
Graph store	Neo4j Community in Docker + Graph Data Science plugin	Native path queries for “how did money get from A to B”, plus free community-detection algorithms for mule rings.
Async work	BullMQ job queues on Redis	A trace takes seconds to minutes; you need retries, progress events and per-provider concurrency caps.
Real-time + graph UI	Socket.IO; Cytoscape.js	Rooms per case for live updates; Cytoscape has flow layouts and one-line PNG export for reports.
Documents	HTML templates → Puppeteer PDF, plus JSON	One template renders on screen and in print; JSON is the machine-readable twin.
Demo safety	Record-and-replay of every external API response	Finale Wi-Fi and free-tier throttling cannot be allowed to break the live demo.

Problem–statement coverage

Each of your six modules maps to specific backend and frontend steps, so judges can see every clause of the problem statement land somewhere concrete.

Module	Backend	Frontend	Key free resources
1 • TRON-first multi-chain wallet intake	B2, B3	F2	TronGrid, Tronscan, TronWeb / ethers / bitcoinjs validation
2 • Multi-hop tracing & transaction graph engine	B1, B3, B4	F4	TronGrid, Etherscan V2, MegaNode, Esplora; Neo4j
3 • VASP attribution & mule-wallet clustering	B5, B6	F4, F6	eth-labels, Tronscan tags, OFAC lists, FIU-IND notices, Neo4j GDS
4 • Explainable risk scoring & typology	B7	F5	Elliptic family, Kaggle ETH fraud, TRON bootstrap labels, SHAP
5 • Real-time alerting & LEA dashboard	B8	F3, F7	TronGrid events, Alchemy WebSocket, mempool.space WebSocket
6 • Freeze notice & standardized report export	B9, B10	F8	Puppeteer, SHA-256, SAHYOG/NCRP adapter mocks

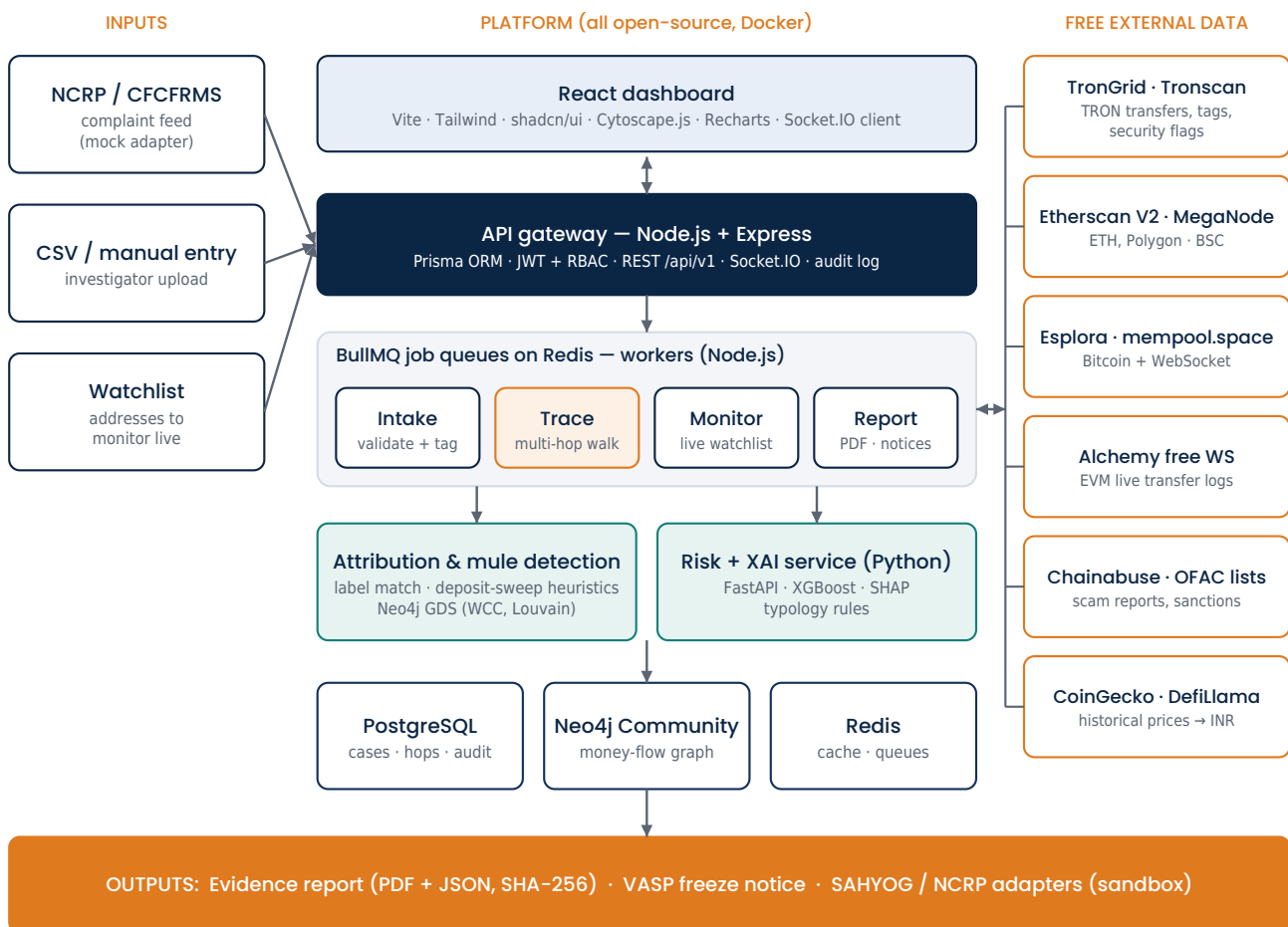
Back your TRON-first claim with a source

Your pitch says most Indian fraud settlements land on TRON/USDT-TRC20. It is a strong and widely repeated observation, but judges from I4C will know the numbers. Put a cited figure on the slide (an I4C/MHA statement, a Chainalysis or TRM Labs report, or your own count from the synthetic-but-realistic complaint set) rather than stating it as fact.

SECTION 2

2 · System architecture

Three columns: where cases come from, the platform itself, and the free external data it pulls. Everything in the middle column runs from one docker-compose file on a laptop.



Life of a complaint

1. A complaint arrives from the mock NCRP feed or a CSV upload. The **Intake** worker cleans, validates and chain-tags every address and transaction hash.
2. TRON seeds skip generic probing and go straight into the TRON trace queue with the USDT-TRC20 contract pre-selected — this is the TRON fast path.
3. The **Trace** worker expands hops through chain adapters (cached and rate-limited), writes each hop to PostgreSQL and Neo4j, and streams `trace.hop` events to the browser.
4. Every newly seen address is checked against the label store: VASP registry, OFAC lists, Tronscan flags, Chainabuse reports.
5. The attribution engine runs its heuristics on the growing graph; the mule detector runs rules plus Neo4j community detection.
6. The risk service scores each address and returns its top SHAP reasons and a fraud-typology tag.

7. Mule and frontier addresses join the watchlist; the **Monitor** worker watches them live on each chain.
8. When tainted funds reach an exchange-attributed address, a CRITICAL alert fires and a freeze notice is auto-drafted.
9. The investigator reviews, a supervisor approves, and the **Report** worker exports PDF + JSON and submits through the SAHYOG/NCRP adapter (sandbox).

SECTION 3

3 · Free APIs and providers

Get every key on day one — some need email verification. Store them only in `.env` and never commit them. Quotas below are what was published around September 2026; read live limits from each console and never hard-code them into business logic.

Provider	What you get	Access	Free-tier catch	Steps
TronGrid	TRON: TRX and TRC-20 (USDT) transfer history per account, contract events, full-node API	Free key, sent as a request header	Daily quota plus per-second cap; historically 100K requests/day per account. 429/403 on overrun — back off with jitter.	B3 B4 B8
Tronscan API	TRON account detail and public name tags; Security Service: fraud-transaction flag, stablecoin-blacklist flag, same-tail spoofing check, risky approvals	Free key, same header name	Unkeyed calls are throttled hard; cache every response.	B3 B5 B7
Etherscan API V2	One key for Ethereum, Polygon, Arbitrum and most EVM chains via a chainid parameter	Free key	About 5 calls/s and 100K/day. Since late 2025 the free tier no longer serves BNB Chain, Base or Optimism.	B3 B4
NodeReal MegaNode (BSCTrace)	BNB Smart Chain transaction history and JSON-RPC	Free-tier key	BNB Chain's recommended replacement after BscScan API deprecation.	B3
Blockscout PRO API	Fallback explorer API across 100+ chains	Free key	5 req/s, 100K credits/day on free tier.	B3
Blockstream Esplora / mempool.space	Bitcoin addresses, transactions, UTXO outspends; mempool.space WebSocket address tracking	No key	Public instances are rate-limited; both are open-source and self-hostable.	B3 B4 B8
Alchemy (or Infura)	EVM JSON-RPC + WebSocket subscriptions for live Transfer logs	Free key	Compute-unit quota; subscribe narrowly by topic.	B8
Chainabuse API	Scam reports by address, filterable by chain, category and moderator-checked status	Free key from your profile	Evidence and reporter contact data are not exposed publicly.	B5 B7
OFAC SDN list + 0xB10C lists	Sanctioned addresses per asset, including TRX, USDT, ETH, BTC and BSC	Public download	GitHub lists are rebuilt nightly from the Treasury XML.	B5 B7 B8
CoinGecko Demo API	Historical prices, including direct INR	Free demo key	Historical range is limited on the free plan.	B3 B9
DefiLlama (free endpoints)	Historical USD prices; protocol list with bridge and DEX categories	No key	The dedicated bridges endpoints moved behind the paid plan in 2025 — use open bridge contract lists instead.	B3 B4
Frankfurter	Daily USD→INR reference rates (ECB)	No key	Fallback when CoinGecko has no INR point.	B9
Chainstack / Ankr free TRON RPC	Backup TRON node access	Free key	Use as failover if TronGrid throttles during the demo.	B3

Reality check before you promise anything

No public government APIs. NCRP/CFCFRMS and SAHYOG do not expose public APIs. Write an OpenAPI spec for the payloads you expect, run a mock server from it, and code against an adapter interface.

Free tiers shift. Etherscan dropped BNB Chain from its free plan; DefiLlama put bridges behind a paywall. Keep one provider interface per chain so swapping a provider touches one file.

TRON has no push subscriptions on public providers. Poll per block (about 3 s block time) or stream the USDT contract's events; design B8 around that.

Say “evidence-grade”, not “court-admissible”. Admissibility is decided by a court. You can design for it: hashes, timestamps, sources, methodology, and a certificate template.

SECTION 4

4 · Free and public datasets

Datasets play three roles: **training and benchmarking** the risk model, **lookup** labels during a live trace, and **demo data**. Keep a source and confidence column on every label you import — the evidence report cites them.

Dataset	What it contains	Access	Used for
Elliptic Data Set	About 204K Bitcoin transaction nodes, 166 features, 234K payment edges; roughly 2% labeled illicit and 21% licit	Kaggle (free login)	Benchmark your graph + tree models against the published baseline
Elliptic2	122K labeled subgraphs inside a background graph of 49M node clusters and 196M edges — the “shapes” of laundering	Download via official GitHub guide	Subgraph patterns for mule-ring detection; work on a sample, the full set is heavy
Elliptic++	Bitcoin transactions plus a wallet-address (actor) graph with labels	GitHub (git-disl)	Address-level illicit classification benchmark
Kaggle Ethereum Fraud Detection	Account-level features (timing, unique counterparties, ERC-20 stats) with a fraud flag	Kaggle	EVM risk model; its feature ideas port directly to TRON
eth-labels	Public labeled addresses (exchanges, bridges and more) for Ethereum and many EVM chains; actively maintained	GitHub + local API	Exchange hot-wallet seeds for attribution
OFAC sanctioned addresses	SDN digital-currency addresses per asset, incl. TRX and USDT	GitHub lists branch	Hard CRITICAL override; sanctions-exposure feature
USDT-TRC20 blacklist events	Addresses frozen by Tether on TRON (AddedBlackList events on the USDT contract)	TronGrid events API	High-confidence risk labels for the TRON model
Tronscan security flags	Per-address fraud-transaction, fraud-token-creator and stablecoin-blacklist flags	Tronscan API	Label bootstrap + live risk factor
Chainabuse reports	Victim and investigator scam reports by chain and category	API (free key)	Weak labels, typology hints
ScamSniffer / MetaMask phishing lists	Phishing domains and, where published, drainer addresses (EVM)	GitHub	Phishing/drainer typology signals
FIU-IND releases & VDA SP list	Registration status; non-compliance notices (9 offshore exchanges in Dec 2023; 15 providers on 9 Sep 2026)	fiuindia.gov.in / PIB	VASP registry jurisdiction and status chips
Synthetic NCRP complaints	Faker-generated complaints seeded with real flagged on-chain addresses; no real victim data	Your repo	Demo, load tests, intake validation

Build the TRON label set yourself (one day of work)

TRON has no Elliptic-style public labeled dataset, which is exactly why a TRON-first tool is interesting. Bootstrap one:

- Pull every AddedBlackList event from the USDT-TRC20 contract TR7NHqjEhKQxGTCi8q8ZY4pL8otSzgJLj6t via TronGrid's events endpoint.
- Add OFAC TRX and USDT addresses, and Chainabuse TRON reports with `checked=true`.
- Enrich each address with Tronscan security flags and account age.
- Negatives: sample active USDT holders with none of these flags, excluding tagged services (exchanges, contracts).
- Label “high-risk”, not “criminal” — Tether also freezes wallets holding stolen funds. Split train/test by time to avoid leakage.

Why Elliptic is a benchmark, not your training set

Elliptic's 166 features are anonymised, so a model trained on them cannot be applied to a TRON address. Use the Elliptic family to prove your graph-feature and GNN approach works (and to quote a benchmark), then train the production model on the address features in Appendix B, computed from your own traces.

SECTION 5

5 · Backend implementation (B0–B11)

Build in this order. Each step leaves the system runnable, so you always have something to demo.

B0 Project foundation and environment

Goal. one command brings the full stack up on every teammate's laptop.

- Monorepo with `apps/web`, `apps/api`, `workers`, `services/ml`, and `packages/shared` (types, address utils, constants).
- `docker-compose.yml`: `postgres:16`, `neo4j:5-community` with `NEO4J_PLUGINS=["graph-data-science"]`, `redis:7`, `api`, `workers`, `ml`, `web`.
- `.env.example` listing every key from Section 3; a `/health` endpoint that pings Postgres, Neo4j, Redis, ML and each provider.
- GitHub Actions: lint, type-check and unit tests on every pull request.
- Fixture recorder: an axios interceptor that saves each external response to `fixtures/<provider>/<sha1(url)>.json`. Mode switch `DATA_MODE=live|record|replay`.

Uses Docker, pnpm workspaces, GitHub Actions (free minutes)

✓ **Done when** docker compose up gives an all-green `/health`, and replay mode works with Wi-Fi off.

B1 Data model: PostgreSQL for workflow, Neo4j for money flow

Goal. relational tables for cases and evidence; a graph for paths and communities.

- Prisma models: `User`, `Complaint`, `ComplaintAddress`, `Case`, `TraceJob`, `Hop`, `AddressProfile`, `Label`, `Vasp`, `VaspAddress`, `RiskScore`, `Alert`, `WatchlistItem`, `Report`, `FreezeNotice`, `AuditLog`.
- Neo4j: `(:Address {chain, addr})`, `(:Entity {name, type})`, `[[:TRANSFER {tx, token, amount, usd, ts}]]`, `[[:BELONGS_TO {heuristic, confidence}]]`, with a uniqueness constraint on `(chain, addr)`.
- Store amounts as `Decimal` or strings (never floats); store UTC, display IST.

```

model Complaint {
  id          String    @id @default(cuid())
  ackNo       String    @unique           // NCRP acknowledgement number
  reportedAt  DateTime
  category    String
  amountInr   Decimal
  network     String?   // "TRC20" | "ERC20" | "BEP20" as stated by victim
  addresses   ComplaintAddress[]
  caseId      String?
}

model Hop {
  id          String    @id @default(cuid())   traceId String  hopNo Int  chain String
  txHash      String    fromAddr String  toAddr String  token String  amount Decimal  usd Decimal?
  ts          DateTime
  @@unique([traceId, txHash, fromAddr, toAddr])
}

// Neo4j – idempotent write for each hop (batched with UNWIND, 500 rows per call)
UNWIND $hops AS h
MERGE (a:Address {chain: h.chain, addr: h.from})
MERGE (b:Address {chain: h.chain, addr: h.to})
MERGE (a)-[t:TRANSFER {tx: h.tx, idx: h.idx}]->(b)
SET t.token = h.token, t.amount = h.amount, t.usd = h.usd, t.ts = h.ts

```

Uses Prisma, neo4j-driver

✓ **Done when** a seed script loads one sample case and the same hops are queryable in both stores.

B2

Complaint intake and chain auto-detection (TRON fast path)

Goal. turn messy complaint input into validated, deduplicated, chain-tagged seeds in milliseconds.

- Endpoints: POST /complaints/import (CSV streamed through csv-parse), POST /complaints (JSON), and a poller against the mock NCRP feed.
- Normalise: trim, strip zero-width characters and spaces, fix 0x → 0x. Never auto-correct Base58 characters — flag them instead.
- Detection order: TRON first (regex + Base58Check), then EVM (0x + EIP-55), then Bitcoin (Base58 / bech32). Also detect transaction hashes. Rules in Appendix A.
- TRON fast path:** TRON seeds bypass multi-chain probing and are enqueued directly on the TRON trace queue with USDT-TRC20 pre-selected.
- EVM ambiguity: one 0x address is valid on ETH, BSC and Polygon. Use the complaint's network field (TRC20/ERC20/BEP20) first; otherwise probe all three in parallel and keep chains with activity.
- Fake-token guard: only trust official stablecoin contracts (e.g. USDT-TRC20 TR7NH...jLj6t, USDT-ERC20 0xdAC1...1ec7, BSC-USD 0x55d3...7955). TRON is full of look-alike “USDT” tokens used in address poisoning.
- Deduplicate across complaints; the same address in two complaints auto-links the cases (crime linkage).

```

const RULES = [ // checked in this order: TRON first
  { chain: 'TRON', re: /^T[1-9A-HJ-NP-Za-km-z]{33}$/ , verify: isTronAddress }, // Base58Check
  { chain: 'EVM', re: /^0x[a-fA-F0-9]{40}$/ , verify: ethers.isAddress }, // EIP-55
  { chain: 'BTC', re: /^(bc1[ac-hj-np-z02-9]{11,71}|[13][1-9A-HJ-NP-Za-km-z]{25,34})$/ ,
    verify: isBtcAddress }, // bitcoinjs-lib
];

```

Uses TronWeb, ethers v6, bitcoinjs-lib, zod, multer, csv-parse

✓ **Done when** a 1,000-row CSV validates in under 2 seconds with a per-row error report, and TRON rows reach the trace queue first.

B3 Chain adapters and the provider layer

Goal. one interface, many chains, resilient to free-tier limits.

- **TronAdapter:** TronGrid `/v1/accounts/{addr}/transactions/trc20` (USDT first, paginate with `meta.fingerprint`) and `/v1/accounts/{addr}/transactions` for TRX; Tronscan for account detail, public tags and security flags; earliest inbound transfer gives the account's activator.
- **EvmAdapter:** Etherscan V2 `txlist`, `tokenTx`, `txlistinternal` with `chainid=1` (Ethereum) or `137` (Polygon); BSC through MegaNode; Blockscout as fallback.
- **BtcAdapter:** Esplora `/address/:a/txs` and `/tx/:id/outspends` (which transaction spent each output — the key call for forward tracing); mempool.space as mirror.
- Provider guard: token-bucket limiter per provider (bottleneck), Redis cache (confirmed transactions cached forever, account metadata for 1 hour), retries with exponential backoff and jitter on `429/403/5xx`, circuit breaker that fails over to the backup provider.
- Pricing: CoinGecko Demo (INR direct) → fallback DefiLlama historical USD × Frankfurter USD→INR, always at the transaction's timestamp.

```
type Chain = 'TRON' | 'ETH' | 'BSC' | 'POLYGON' | 'BTC';
interface Transfer {
  chain: Chain; txHash: string; idx: number; from: string; to: string;
  token: string; amount: string; usd?: number; ts: number; block: number; // amount as string
}
interface ChainAdapter {
  getTransfers(addr: string, dir: 'in' | 'out', o: { since?: number; cursor?: string })
    : Promise<{ items: Transfer[]; next?: string }>;
  getAccountMeta(addr: string): Promise<AccountMeta>; // createdAt, activator, flags, public tag
}
// TronGrid call (USDT-TRC20 outgoing, confirmed only)
const r = await http.get(`https://api.trongrid.io/v1/accounts/${addr}/transactions/trc20`, {
  headers: { 'TRON-PRO-API-KEY': env.TRONGRID_KEY },
  params: { limit: 200, contract_address: USDT_TRC20, only_confirmed: true, only_from: true,
    min_timestamp: sinceMs, fingerprint: cursor } });
```

Uses	TronGrid, Tronscan, Etherscan V2, MegaNode, Blockscout, Esplora/mempool.space, CoinGecko, DefiLlama, Frankfurter
✓ Done when	paging 1,000+ transfers of a busy TRON address never leaks a 429 to the caller, and replay mode returns identical results offline.

B4 Multi-hop tracing engine

Goal. follow victim funds outward until they reach an exchange, mixer, bridge or stop condition, persisting every hop.

- BullMQ trace queue, one job per seed; progress and each new hop streamed over Socket.IO.
- Expand highest tainted value first (priority queue), not plain breadth-first — investigators care about where most of the money went.
- Taint model: proportional (“haircut”) by default, FIFO as an option; the report states which model was used.
- Stop conditions: labeled service reached (VASP, mixer, bridge, sanctioned); max hops (default 6); min value (e.g. USD 10); time window (30 days after the incident); top 10 outflows per node; any address with more than 5,000 counterparties is treated as an unlabeled service and flagged for review.
- **DEX swaps:** pair Transfer events inside the same transaction and continue with the output token. **Bridges:** match open bridge contract lists, look the destination transaction up in the bridge's public explorer, add a cross-chain edge, continue on the destination chain. **Mixers:** stop, label, and mark taint as uncertain beyond this point.
- Write hops to Postgres and Neo4j in batches; a re-run of the same trace is served from cache and the graph store.

```

push(seed, hop = 0, taint = reportedAmount)
while queue not empty and budget left:
    node = pop_max_taint()
    if isService(node):                                # VASP, mixer, bridge, sanctioned
        record_terminal(node)
        if isBridge(node): push(resolveDestination(node)) # continue on the other chain
        continue
    if node.hop >= MAX_HOPS: mark_frontier(node); continue
    outs = adapter.getTransfers(node.addr, 'out', since = node.firstTaintedAt)
    outs = top_k(filter(outs, value >= MIN_VALUE and within WINDOW), k = 10)
    for o, share in allocate_taint(node.taint, outs, model = HAIRCUT):
        persist_edge(node, o); emit('trace.hop', o); push(o.to, node.hop + 1, share)

```

Uses BullMQ, Redis, Neo4j, all chain adapters

✓ **Done when** a 6-hop TRON trace of about 200 edges finishes in under a minute live (seconds in replay) and reopens instantly.

B5 VASP attribution and exchange-wallet clustering

Goal. name the nearest exchange with a confidence score and an evidence trail an officer can read.

- Label store merges eth-labels, Tronscan public tags, OFAC lists, Chainabuse and manual investigator labels, each with source, confidence and fetched-at time.
- VASP registry: name, type (centralised exchange, instant-swap, OTC, P2P), jurisdiction, FIU-IND status with date and source, law-enforcement contact or portal, known hot wallets per chain.
- **H1 · Deposit-address sweep** (EVM and TRON): an address that receives from unrelated senders and forwards at least 95% to a known hot wallet within 24 hours is that exchange's deposit address (the deposit-address heuristic from Victor, 2020).
- **H2 · TRON activation and resources:** the account was activated (first TRX received) or supplied with energy by a known exchange fee or activator wallet.
- **H3 · Bitcoin common-input ownership:** union-find clustering over multi-input transactions, skipping CoinJoin-like transactions (many equal-value outputs).
- **H4 · Direct label hit** on the address or its cluster.
- Combine heuristic scores with noisy-OR into one confidence; store every fired heuristic with its transaction links as evidence.
- Tag instant-swap services separately — three of the fifteen providers FIU-IND noticed on 9 September 2026 (ChangeNow, SimpleSwap, FixedFloat) are swap services, a common no-account exit route.

Uses eth-labels, Tronscan, OFAC lists, Chainabuse, FIU-IND releases, Neo4j

✓ **Done when** on a held-out set of labeled deposit addresses, the correct VASP ranks first for at least 80% (your target — measure and report it honestly).

B6 Mule and layering detection

Goal. surface the intermediary wallets and rings that obscure the trail.

- Compute per-address features from hops and account metadata (Appendix B).
- Rules: pass-through (value out $\approx$ value in, median dwell under 60 minutes); fan-out split (5+ outputs within an hour of an inbound); peel chain (Bitcoin); fresh wallet (under 7 days old at first taint); TRX-dust USDT wallet (holds almost no TRX, relies on rented energy).
- Neo4j GDS on the case subgraph: weakly connected components and Louvain to find rings; degree and betweenness to find collector wallets.
- Cross-case linkage: any wallet appearing in two or more complaints gets a shared-mule flag and links the cases.

Uses Neo4j Graph Data Science (community edition)

✓ **Done when** the graph view can highlight mule-ring communities and each mule lists the rules that fired.

B7 Risk scoring, typology and XAI service (Python)

Goal. a 0–100 score with its top reasons in plain language, plus a fraud-typology tag.

- FastAPI endpoints: POST /score, POST /typology, GET /model-card.
- Training: TRON bootstrap labels (Section 4) and the Kaggle Ethereum fraud set; Elliptic family as benchmark only.
- Model: XGBoost (or LightGBM) with class weights, time-based split, isotonic calibration.
- Hybrid score = $0.6 \times \text{calibrated ML probability} \times 100 + 0.4 \times \text{rule score}$. Hard overrides: sanctioned or stablecoin-blacklisted → CRITICAL. Bands: 0–29 Low, 30–59 Medium, 60–79 High, 80–100 Critical.
- SHAP TreeExplainer → top positive contributions mapped to sentences an officer can read. Show 3–5, never a raw feature name.
- Typology: rules on on-chain signals plus the NCRP complaint category as a prior (table below).
- Publish a model card: data sources, metrics, known limitations, date trained.

```
REASONS = {
    "dwell_median_min": lambda v: f"Forwards funds a median {v:.0f} min after receiving them",
    "fan_out_1h": lambda v: f"Split incoming funds across {int(v)} wallets within an hour",
    "age_at_taint_days": lambda v: f"Wallet was {v:.0f} days old when it received victim funds",
    "shared_mule_cps": lambda v: f"Shares {int(v)} counterparties with a known mule ring",
}
sv = explainer(X_row) # shap.TreeExplainer(xgb_model)
top = sorted(zip(cols, sv.values[0], X_row.iloc[0]), key=lambda t: -abs(t[1]))[:5]
factors = [{"feature": f, "impact": round(float(s), 3), "reason": REASONS[f](v)}
            for f, s, v in top if s > 0]
```

Typology	On-chain signals	Complaint prior
Investment / pig-butchering	Escalating deposits from the same victim over days or weeks; a consolidation wallet; exit via OTC, instant-swap or exchange	Investment or trading fraud
Sextortion	Many small, similar-value inbound payments from unrelated senders to a short-lived address	Sextortion or extortion
Ransomware	Bitcoin-heavy; one large inbound per victim; fast peel chains; mixer or privacy-coin swap exposure	Ransomware
Phishing / wallet drainer	Victim approval or permission change right before the outflow (Tronscan approval and multi-sign checks; EVM Approval events); known drainer contract	Phishing, fake app
Address poisoning	Zero-value or dust transfers from look-alike addresses (Tronscan flags same-tail spoofing)	Wrong-address transfer
Uses	XGBoost, scikit-learn, SHAP, pandas, FastAPI, Tronscan security API	
✓ Done when	every scored wallet returns a score, band, at least 3 readable reasons and a typology with confidence in under 200 ms.	

B8 Real-time monitoring and alerts

Goal. tell investigators the moment suspect funds move — above all when they reach an exchange.

- Watchlist fills automatically from traces (mules and frontier addresses) plus manual adds from the graph view.
- TRON:** no push subscription, so poll. Option A streams USDT-TRC20 Transfer events from TronGrid's events endpoint and filters against a Redis set of watched addresses in O(1). Option B polls watched addresses adaptively (hot every 30 s, warm every 5 min, cold hourly). Start with B; measure calls/day against the quota before switching to A.
- EVM:** Alchemy WebSocket `eth_subscribe('logs')` on the USDT Transfer topic with watched addresses as topic filters.
- Bitcoin:** mempool.space WebSocket address tracking.
- Delivery: Socket.IO room per case, persisted Alert rows, optional email through a free SMTP relay.

Rule	Trigger	Severity / action
A1 Movement	A watched wallet sends above a threshold	Medium or High by amount
A2 VASP landing	Tainted funds reach an exchange-attributed address	CRITICAL — “freeze window open”; freeze notice auto-drafted
A3 Obfuscation	Funds enter a mixer, bridge or instant-swap service	High; trace continues on the far side where possible
A4 Linkage	A new complaint touches an existing cluster	High; cases linked for a single investigation
A5 Blacklist	A watched address is USDT-blacklisted or appears in an OFAC update	Info or High; risk score recomputed
Uses	TronGrid events, Alchemy WebSocket, mempool.space WebSocket, Socket.IO, Redis	
✓ Done when	a replayed transfer from a watched TRON wallet appears on the dashboard within 30 seconds, and a VASP landing raises a CRITICAL alert with a drafted notice.	

B9 Evidence report, freeze-notice generator, SAHYOG/NCRP adapters

Goal. one click from finding to a standardised, verifiable document.

- Handlebars HTML templates with print CSS → PDF via Puppeteer; the same data exported as versioned JSON (`evidence.v1.json`).
- Evidence report: case metadata (NCRP acknowledgement, FIR number), victim transaction, hop table (chain, from, to, tx hash, UTC and IST time, token amount, INR value at the time, explorer link), graph snapshot, attribution with confidence and evidence, risk breakdown with reasons, data sources with retrieval times, methodology (taint model, stop conditions), limitations.
- Integrity: SHA-256 of the canonical JSON printed in the PDF footer and as a QR code; a `/verify` endpoint re-computes it. The audit log is hash-chained (each entry stores the previous entry's hash).
- Freeze notice: addressed to the VASP's compliance or law-enforcement contact from the registry; lists deposit addresses, tx hashes, amounts in token and INR, IST timestamps; requests freezing and preservation of KYC and logs. The legal-provision field is edited by the IO and confirmed with the legal cell. Supervisor approval is required before “Send”.
- Certificate template for electronic records under the Bharatiya Sakshya Adhiniyam, 2023 (Section 63) — have a legal advisor validate the wording.
- Adapters: an `IntegrationAdapter` interface with `SahyogAdapter` and `NcrpAdapter` implemented against a mock server generated from your own OpenAPI spec.

Uses	Handlebars, Puppeteer, Node crypto, qrcode, Prism (or a small Express mock)
✓ Done when	PDF and JSON export in under 5 seconds, the hash verifies, and the mock SAHYOG returns a submission ID shown in the UI.

B10 Security, access control and audit

Goal. behave like software a police unit could actually deploy.

- Short-lived JWT access tokens plus refresh tokens; roles Investigator, Supervisor, Admin, Viewer.
- helmet, express-rate-limit, a CORS allowlist, zod validation on every request body.
- Encrypt victim PII fields at the application layer (AES-256-GCM); collect the minimum personal data (DPDP Act principles).
- Audit every case view, export, notice and label change — the audit log doubles as chain of custody.

Uses	jsonwebtoken, helmet, express-rate-limit, zod, Node crypto
✓ Done when	the role matrix is covered by tests and every export has an audit entry.

B11 Testing, performance and demo hardening

Goal. nothing breaks on stage.

- Unit tests (Vitest for Node, pytest for ML); adapter contract tests against recorded fixtures.
- End-to-end test in replay mode: complaint → trace → attribution → alert → report.
- k6 load test on the graph and risk endpoints; a provider-quota panel on the admin page.
- A “golden demo case” seeded in replay mode, plus two backup cases.

Uses Vitest, pytest, k6, GitHub Actions

✓ **Done when** CI is green and the end-to-end replay passes with networking disabled.

SECTION 6

6 · Frontend implementation (F0–F9)

React + Tailwind as your team already uses, with a small set of libraries chosen for this problem. Build the graph explorer (F4) early — it is the screen judges remember.

F0 App shell and design system

Goal. a consistent, fast shell every screen plugs into.

- Vite + React + TypeScript + Tailwind + shadcn/ui; React Router; TanStack Query for server state; Zustand for UI state; lucide-react icons.
- Operations-console theme: dark by default with a light, print-friendly mode; tokens for risk bands and chain colours.
- Layout: left navigation (Dashboard, Intake, Cases, Alerts, Watchlist, VASP registry, Reports, Settings); top bar with global address search and an alert bell.
- API client with interceptors for JWT refresh and consistent error toasts.

Uses Vite, Tailwind, shadcn/ui, TanStack Query, Zustand

✓ **Done when** routing, auth guard and the API client work end to end against the B0 stack.

F1 Login and role-aware UI

Goal. the interface only offers actions the user's role allows.

- Login form with react-hook-form + zod.
- Role-gated components — only a Supervisor sees “Approve and send notice”; Viewers get read-only case pages.

✓ **Done when** each role sees a different, correct set of actions.

F2 Complaint intake

Goal. make bulk and single intake painless, and show the TRON fast path visibly.

- Drag-and-drop CSV parsed with papaparse in a Web Worker so large files never freeze the page; paste box for raw addresses; single-complaint form (NCRP acknowledgement, date, amount in INR, category, network).
- Live preview table with chain badges (TRON, ETH, BSC, POLYGON, BTC), inline validation errors, duplicate and linked-case markers.
- “Start tracing” with advanced options: max hops, minimum value, time window, taint model.

Uses papaparse, react-hook-form, zod

✓ **Done when** a 1,000-row file previews without UI lag and TRON rows are visibly prioritised.

F3 Command dashboard

Goal. give a unit head the whole picture in five seconds.

- KPI cards: open cases, value traced (INR), VASPs identified, freeze windows open, median time-to-attribution.
- Recharts panels: traces per day, typology mix, top destination VASPs, chain split.
- Live alert feed and a recent-cases table (TanStack Table with sort, filter, pagination).

Uses Recharts, TanStack Table, Socket.IO client

F4 Fund-flow graph explorer (hero screen)

Goal. let an officer see and explore where the money went.

- Cytoscape.js with a left-to-right flow layout (dagre) and fcose for dense clusters.
- Visual encoding: node shape by role (victim, wallet, VASP, mixer, bridge); fill by risk band; border by chain; edge width by log of value; dashed edges for cross-chain hops. Never rely on colour alone — add icons and labels.
- Interactions: click for side panel; expand a node on demand (lazy-load the next hop); filter by chain, value and time; time-slider replay of the flow; “heaviest path” highlight; lasso-select to add to the watchlist; right-click to copy the address or open the explorer.
- Live mode: subscribe to `trace.hop` and animate nodes in while the trace runs.
- Scale: collapse mule communities into compound nodes above ~1,500 elements; export PNG for the report.

Uses cytoscape, react-cytoscapejs, cytoscape-dagre, cytoscape-fcose

✓ **Done when** a 1,000-node case stays smooth to pan and zoom, and the exported PNG drops straight into the report.

F5 Wallet profile and XAI panel

Goal. answer “why is this wallet risky?” in one glance.

- Score gauge and band; “Why this wallet is risky” — horizontal bars of SHAP contributions labeled with the plain-language reasons.
- Flags with source and fetch time (sanctioned, stablecoin-blacklisted, Chainabuse reports); typology chip with confidence.
- Activity timeline (in/out over time), counterparties table, deep links to Tronscan, Etherscan or mempool.space.
- Human-in-the-loop: “Confirm” or “Dispute” a label; feedback goes to the label store and future training.

Uses Recharts

✓ **Done when** every reason shown traces back to a feature value the officer can verify on an explorer.

F6 VASP attribution view

Goal. make the “where do we send the notice?” answer unmissable.

- One card per destination VASP: name, type, jurisdiction, FIU-IND status chip, amount reached, hop count, confidence.
- Evidence list: which heuristic fired, with transaction links; a mini path graph from victim to deposit address.
- Primary action: “Draft freeze notice”.

F7 Real-time alerts center

Goal. no freeze window goes unnoticed.

- Socket.IO client; toast plus sound for CRITICAL; alerts table with severity and status filters.
- Acknowledge, assign and snooze; deep link to the exact hop in the graph.

F8 Report and freeze-notice builder

Goal. from finding to signed-off document without leaving the case.

- Form for IO name, rank, police station, FIR number, NCRP acknowledgement and legal-provision fields.
- Live PDF preview, PDF and JSON download, SHA-256 display with a “Verify” button.
- Workflow: Draft → Pending approval → Approved → Sent; “Submit to SAHYOG (sandbox)” with status tracking.

✓ **Done when** a supervisor can approve and submit to the mock SAHYOG in under a minute.

F9 Polish

Goal. look and feel like a finished product.

- Responsive layout, keyboard navigation, colour-blind-safe palette.
- Hindi/English toggle with react-i18next — a thoughtful touch for state police users.
- IST formatting everywhere, proper empty, loading and error states, and a visible “Demo mode (replay)” banner.

SECTION 7

7 · API contract between frontend and backend

Agree on this in week one and mock it (MSW in the browser) so frontend and backend can build in parallel. All routes sit under `/api/v1` and require a JWT except login.

Method & route	Purpose	Step
POST /auth/login · POST /auth/refresh	Sign in, rotate tokens	B10 / F1
POST /complaints/import · POST /complaints · GET /complaints	Bulk CSV, single complaint, list and filter	B2 / F2
POST /cases/:id/traces	Start a trace {seeds, maxHops, minValue, windowDays, taintModel}	B4 / F2
GET /traces/:id · GET /traces/:id/graph	Status and progress; graph JSON with filters (chain, minValue, time)	B4 / F4
GET /addresses/:chain:addr	Profile: labels, flags, meta, counterparties	B3 / F5
GET /addresses/:chain:addr/risk	Score, band, SHAP reasons, typology	B7 / F5
GET /traces/:id/attribution	Destination VASPs with confidence and evidence	B5 / F6
GET · POST · DELETE /watchlist	Manage monitored addresses	B8 / F4
GET /alerts · PATCH /alerts/:id	List; acknowledge, assign, snooze	B8 / F7
POST /cases/:id/reports?format=pdf json · GET /verify/:hash	Generate evidence report; verify integrity	B9 / F8
POST /cases/:id/freeze-notices · POST /freeze-notices/:id/approve	Draft, approve and send notices	B9 / F8
POST /integrations/sahyog/submit · POST /integrations/ncrp/sync	Adapter calls (sandbox until access)	B9
GET · POST /vasps	VASP registry (Admin can edit)	B5

Socket.IO events

Event	Payload	Consumer
trace.progress	{traceld, hopsDone, frontier, apiCalls}	Progress bar (F2, F4)
trace.hop	{traceld, edge, fromNode, toNode}	Live graph animation (F4)
trace.completed	{traceld, terminals, durationMs}	Case page, attribution view
alert.new	{id, rule, severity, caseld, address, amount}	Toasts, alert center (F7)

Event	Payload	Consumer
report.ready	{reportId, sha256, url}	Report builder (F8)

ML service (internal)

```
POST /score
request { "addresses": [ { "chain": "TRON", "addr": "T...", "features": { ... } } ] }
response [ { "addr": "T...", "score": 84, "band": "Critical",
            "factors": [ { "feature": "...", "impact": 0.21, "reason": "..." } ],
            "overrides": ["STABLECOIN_BLACKLIST"], "modelVersion": "tron-xgb-v1" } ]

POST /typology
request { "caseFeatures": { ... }, "complaintCategory": "sextortion" }
response { "label": "sextortion", "confidence": 0.78, "signals": ["many small similar inbound"] }
```

SECTION 8

8 · Team split and six-week timeline

Six people, six clear lanes. Pairings mark the interfaces where two owners must agree early.

Who	Role	Owns	Pairs with
P1	Backend lead	Express API, Prisma schema, auth/RBAC, audit, report and notice generation, integration adapters (B0, B1, B9, B10)	P5 (API contract)
P2	Chain engineer — TRON specialist	Chain adapters, provider guard, tracing engine, real-time monitor (B3, B4, B8)	P3 (graph writes)
P3	Graph & attribution engineer	Neo4j model, label loaders, VASP registry, attribution heuristics, mule detection (B5, B6)	P4 (features)
P4	ML & XAI engineer	Datasets, TRON label bootstrap, feature pipeline, XGBoost, SHAP, typology, model card (B7)	P5 (XAI panel)
P5	Frontend lead	App shell, graph explorer, wallet XAI panel, attribution view (F0, F4, F5, F6)	P1, P2
P6	Frontend + DevOps + demo	Intake, dashboard, alerts, report builder; Docker, CI, fixtures, pitch deck (F1-F3, F7-F9, B11)	Everyone

Wk	Backend	ML / data	Frontend	Checkpoint
1	B0 stack, B1 schema, API contract, all keys obtained	Download datasets; write label loaders (OFAC, eth-labels)	F0 shell, F1 login, MSW mocks	docker compose up is green for all six
2	B2 intake, B3 TRON adapter, fixture recorder	USDT blacklist pull; Tronscan enrichment	F2 intake with chain badges	CSV → validated TRON seeds
3	B4 tracing (TRON, then EVM, BTC), Neo4j writes	Feature pipeline v1 from traces	F4 graph explorer v1 with live hops	First end-to-end trace on screen
4	B5 attribution, B6 mule detection	Train XGBoost, SHAP, typology rules	F5 XAI panel, F6 attribution view	A VASP named with evidence
5	B8 monitor + alerts, B9 reports and notices, adapter mocks	Model card; evaluation numbers	F3 dashboard, F7 alerts, F8 report builder	Full flow: complaint → notice PDF
6	B10 security, B11 tests and hardening	Freeze model; benchmark table	F9 polish, Hindi toggle	Replay demo passes offline; deck done

For the finale

Freeze scope a week before. Arrive with everything working in replay mode and use the hackathon hours for polish, live-mode runs, and changes the judges ask for. Keep a short list of stretch features (Hindi UI, a second backup provider, email alerts) to pull in if you are ahead.

SECTION 9

9 · Demo script and metrics

Five-to-seven-minute demo

- Intake (30 s).** Upload five synthetic complaints; chain badges appear — three TRON, one ETH, one BTC — and TRON rows jump the queue.
- Live trace (60 s).** Start the TRON case; hops stream into the graph in real time.
- Mules and XAI (60 s).** A mule ring lights up; click a mule and read out its reasons (split across 7 wallets within the hour, 2-day-old wallet, shares counterparties with a known ring).
- Attribution (45 s).** Funds land at a deposit address attributed to an exchange with high confidence; show the evidence and its FIU-IND status chip.
- Freeze window (30 s).** A replayed movement fires the CRITICAL “freeze window open” alert.
- Notice and report (60 s).** Generate the freeze notice and evidence PDF, show the SHA-256 and verify it, submit to SAHYOG (sandbox).
- Crime linkage (30 s).** A new complaint auto-links to the same mule cluster — one investigation instead of two.

Use real on-chain addresses from public flagged lists so the trace is genuine, but only synthetic complaint and victim data.

Metrics judges will ask about

Metric	How you measure it	Target to aim for
Time-to-attribution	Complaint submitted → VASP named, TRON, ≤ 6 hops	Under 2 minutes live
Attribution accuracy	Top-1 VASP on held-out labeled deposit addresses	≥ 80%, reported honestly
Risk model quality	PR-AUC and recall at 90% precision, time-based split; Elliptic benchmark vs the original paper's baseline	Beat the baseline; show the curve
Explainability	Share of scored wallets with ≥ 3 readable reasons	100%
Alert latency	On-chain transfer → dashboard (TRON)	Under 30 seconds
API efficiency	Provider calls per trace; cache hit rate	Cache hit rate above 60% on reruns

SECTION 10

10 · Risks and mitigations

Risk	Impact	Mitigation
Free-tier throttling during the demo	Trace stalls on stage	Redis cache, per-provider limiter, replay mode, backup TRON RPC (Chainstack/Ankr)
No official NCRP/SAHYOG access	“Integration” questioned by judges	Adapter interface + mock server from your own OpenAPI spec; pitch as integration-ready
Sparse TRON labels	Weak model, weak attribution	Label bootstrap (Section 4); investigator Confirm/Dispute feedback loop
Wrong attribution	Notice sent to the wrong exchange	Confidence + evidence on every attribution; notices never auto-send; supervisor approval
Graph explosion at exchange hot wallets	Traces run forever	Stop at labeled services; high-degree cutoff; top-k outflows; time window
Fake USDT tokens and address poisoning	Tracing junk transfers	Official-contract whitelist; ignore zero-value transfers; use Tronscan spoofing flags
EVM chain ambiguity	Tracing the wrong chain	Use the complaint's network field; probe ETH/BSC/Polygon in parallel
Legal wording in notices	Credibility with MHA judges	Editable provision fields; “reviewed by legal cell” checkbox; template vetted by an advisor

Risk	Impact	Mitigation
Victim PII exposure	Privacy breach	Minimise, encrypt, RBAC, audit every access; synthetic data in all demos
Label noise from blacklists	Victims mis-scored as criminals	Label as “high-risk”; show source; human review before action

APPENDIX A

A · Address and hash validation rules

Type	Pattern	Check
TRON address	Starts with T, 34 Base58 characters: ^T[1-9A-HJ-NP-Za-km-z]{33}\$	Base58Check checksum (TronWeb)
EVM address (ETH / BSC / Polygon)	^0x[a-fA-F0-9]{40}\$	EIP-55 checksum when mixed case (ethers.isAddress)
Bitcoin legacy / P2SH	^[13][1-9A-HJ-NP-Za-km-z]{25,34}\$	Base58Check (bitcoins-lib)
Bitcoin bech32 / bech32m	^bc1[ac-hj-np-z02-9]{11,71}\$ (lowercase)	bech32 checksum (bitcoins-lib)
EVM tx hash	^0x[a-fA-F0-9]{64}\$	Exists on chain (probe)
TRON or BTC tx hash	^[a-fA-F0-9]{64}\$ — same shape on both	Try TRON first, then Bitcoin

APPENDIX B

B · Features for the risk model and their reason text

Feature	Definition	Reason shown to the officer
dwel_median_min	Median minutes between receiving and forwarding funds	Forwards funds a median N min after receiving them
fan_out_1h	Distinct recipients within 1 hour of an inbound	Split incoming funds across N wallets within an hour
fan_in_unique	Distinct senders in the last 30 days	Received from N unrelated senders
passthrough_ratio	Value out ÷ value in	Passed on N% of everything it received
age_at_taint_days	Account age when first tainted funds arrived	Wallet was N days old when it received victim funds
activator_label	Label of the wallet that activated this TRON account	Activated by a wallet linked to a known mule cluster
trx_dust_usdt	Holds USDT but almost no TRX; relies on rented energy	Operates as a USDT relay with no TRX of its own
round_amount_ratio	Share of transfers in round amounts	N% of transfers are round amounts
burst_tx_per_hour	Peak transactions per hour	Burst of N transactions in one hour
hops_from_victim / hops_to_vasp	Graph distance on the case trace	N hops from the victim; N hops before an exchange
sanction_exposure	Direct or 1-2 hop exposure to OFAC-listed addresses	Received funds from a sanctioned address
external_flags	Tronscan fraud flag, USDT blacklist, Chainabuse report count	Flagged by Tronscan / blacklisted by Tether / N scam reports
shared_mule_cps	Counterparties shared with known mule rings	Shares N counterparties with a known mule ring
cross_case_count	Complaints this wallet appears in	Appears in N separate complaints

APPENDIX C

C · Repository layout and environment

```

ps26183/
├── apps/
│   ├── web/           React + Vite + Tailwind (F0–F9)
│   └── api/           Express + Prisma: routes, auth, reports, integrations (B1, B2, B9, B10)
├── workers/          BullMQ: intake, trace, monitor, report (B2–B4, B8, B9)
│   └── adapters/      tron.ts, evm.ts, btc.ts, pricing.ts, providerGuard.ts
├── services/ml/       FastAPI: features, train.py, score.py, typology.py, model_card.md (B7)
├── packages/shared/   types, address rules, token whitelist, constants
├── graph/             cypher/ (constraints, GDS queries), seeds/
├── data/              loaders/ (ofac, eth-labels, usdt-blacklist, chainabuse), synthetic/
├── mocks/             openapi/sahyog.yaml, openapi/ncrp.yaml, mock-server/
├── fixtures/         recorded provider responses for replay mode
└── docker-compose.yml

```

```

DATA_MODE=replay          # live | record | replay
TRONGRID_KEY= TRONSCAN_KEY= ETHERSCAN_KEY= MEGANODE_KEY= BLOCKSCOUT_KEY=
ALCHEMY_KEY= CHAINABUSE_KEY= COINGECKO_DEMO_KEY=
DATABASE_URL=postgresql://... NEO4J_URI=bolt://neo4j:7687 REDIS_URL=redis://redis:6379
JWT_SECRET= PII_ENC_KEY= ML_URL=http://ml:8000
TRACE_MAX_HOPS=6 TRACE_MIN_USD=10 TRACE_WINDOW_DAYS=30 TRACE_TOP_K=10 HIGH_DEGREE_CUTOFF=5000

```

APPENDIX D

D · Source links

Resource	Where
TronGrid rate limits	developers.tron.network/reference/rate-limits
Tronscan API and Security Service API	docs.tronscan.org
Etherscan V2 supported chains (free-tier coverage)	docs.etherscan.io/supported-chains
BSCTrace via MegaNode migration guide	bnbchain.org/en/blog/migration-guide-bscscan-api-to-bsctrace-api-via-megano-de
Blockscout PRO API	docs.blockscout.com
Blockstream Esplora API	github.com/Blockstream/esplora/blob/master/API.md
mempool.space API and WebSocket	mempool.space/docs/api
Chainabuse API	docs.trmlabs.com/guides/chainabuse/welcome-to-chainabuse-api
OFAC sanctioned digital-currency address lists	github.com/0xB10C/ofac-sanctioned-digital-currency-addresses
Elliptic Data Set	kaggle.com/datasets/ellipticco/elliptic-data-set
Elliptic2 dataset and paper	github.com/MITIBMxGraph/Elliptic2 · arxiv.org/abs/2404.19109
Elliptic++	github.com/git-disl/EllipticPlusPlus
Ethereum Fraud Detection (Kaggle)	kaggle.com/datasets/vagifa/ethereum-frauddetection-dataset
eth-labels	github.com/dawsbot/eth-labels
ScamSniffer scam database · MetaMask phishing list	github.com/scamsniffer/scam-database · github.com/MetaMask/eth-phishing-detect
DefiLlama API docs	api-docs.defillama.com
Deposit-address clustering heuristic	F. Victor, “Address Clustering Heuristics for Ethereum”, Financial Cryptography 2020
FIU-IND (VDA SP registration and notices)	fiuindia.gov.in